

GESTIÓN DE DATOS DE PAÍSES EN PYTHON: FILTROS,
ORDENAMIENTOS Y ESTADÍSTICAS

INFORME TRABAJO PRÁCTICO PROGRAMACIÓN

Institución: Universidad Tecnológica Nacional - Tecnicatura
Universitaria en Programación (TUP) - Modalidad a Distancia.

Cátedra: Programación I

Integrantes:

Bárcena, Pablo Iván

Costantini, Luciano

Año Lectivo: 2026.

Comisión: 8 y 26

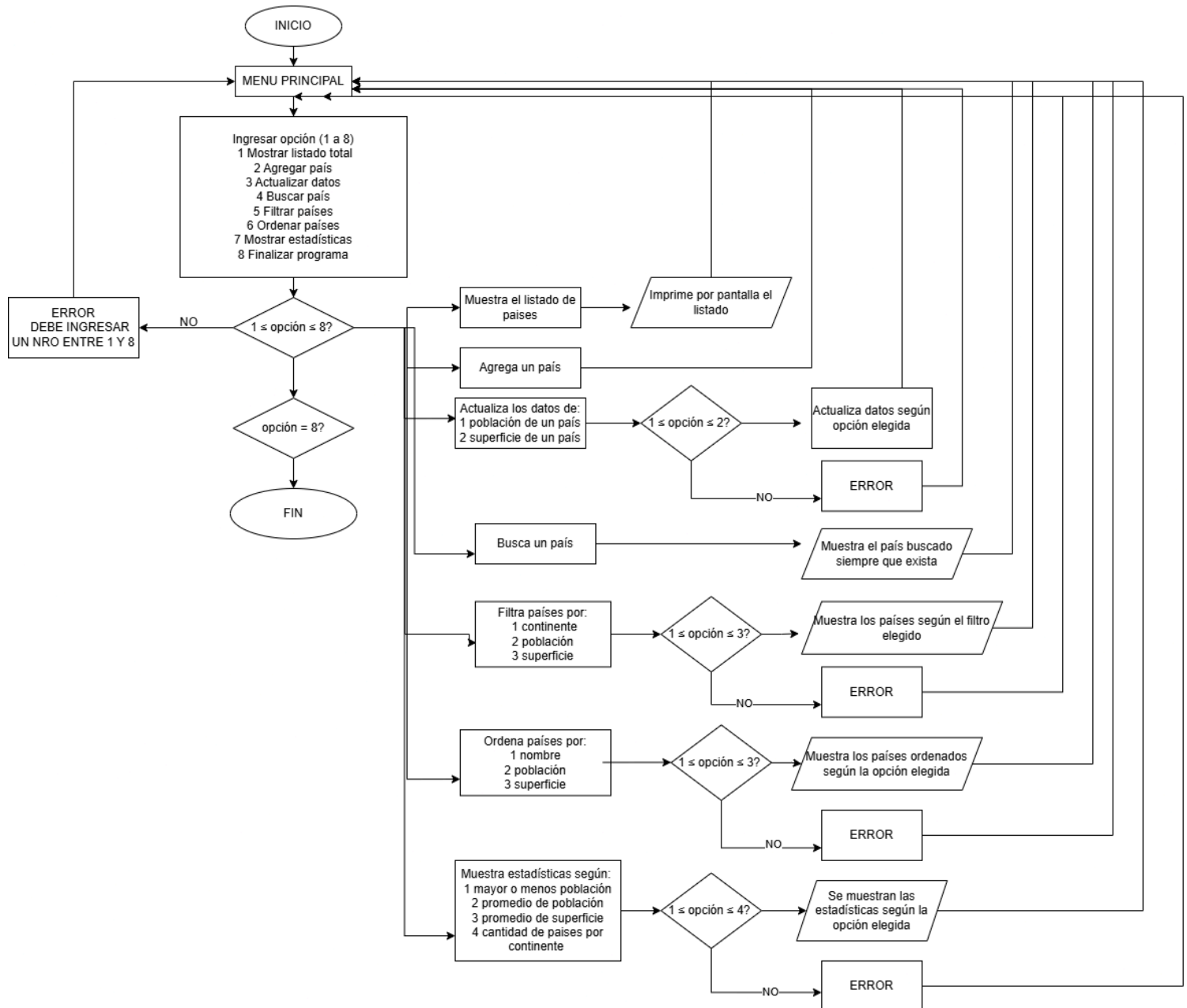
Contenido

1. ENLACES	3
2. DIAGRAMA DEL PROYECTO	3
3. DESARROLLO DEL PROYECTO	4
Descripción general del programa	4
Estructuras de datos utilizadas	4
Lectura de archivos CSV	4
Escritura de archivos CSV	5
Modularización mediante funciones	5
Validación de datos	5
Alta de países	6
Actualización de datos	6
Búsqueda de países	6
Filtros	6
Ordenamiento de datos	7
Estadísticas	8
Manejo de excepciones	8
4. CONCLUSIÓN FINAL	8
5. FUENTES Y BIBLIOGRAFÍA	9

1. ENLACES

REPOSITORIO: <https://github.com/Ivan-cincho/TPI-Barcena-Costantini>

2. DIAGRAMA DEL PROYECTO



3. DESARROLLO DEL PROYECTO

Descripción general del programa

El sistema desarrollado permite gestionar información de países almacenada en un archivo CSV. Las funcionalidades implementadas incluyen la visualización de registros, alta de nuevos países, actualización de datos, búsquedas, filtros, ordenamientos y estadísticas.

La información se almacena en el archivo *países.csv* y se manipula en memoria mediante una lista de diccionarios, donde cada diccionario representa un país con los siguientes atributos:

```
{  
  "nombre": "Argentina",  
  "poblacion": 46044703,  
  "superficie": 2780400,  
  "continente": "America"  
}
```

Estructuras de datos utilizadas

Para el desarrollo se utilizaron principalmente:

- **Listas** para almacenar el conjunto de países.
- **Diccionarios** para representar cada país y sus atributos.
- **Cadenas de texto (str)** para nombres y continentes.
- **Enteros (int)** para población y superficie.

Esta combinación permite acceder fácilmente a los datos mediante claves descriptivas, facilitando la lectura y mantenimiento del código. De esta forma, también, se puede entender más fácilmente el código sin conocerlo de antemano o reutilizar de la misma manera.

Lectura de archivos CSV

La función *cargar_datos()* es la encargada de leer el archivo CSV.

Su funcionamiento consiste en:

1. Abrir el archivo en modo lectura.
2. Leer todas las líneas.
3. Omitir la cabecera.
4. Separar los campos utilizando `split(",")`.
5. Crear un diccionario por cada país.
6. Almacenar cada diccionario en una lista.

Finalmente retorna la lista completa para ser utilizada por el resto de las funciones.

Escritura de archivos CSV

Para almacenar los datos se utiliza la función *guardar_datos()*

Esta función:

1. Abre el archivo en modo escritura.
2. Vuelve a escribir la primera línea del archivo con los nombres de las columnas de los datos.
3. Recorre la lista de países.
4. Convierte cada país al formato CSV mediante *formatear_datos()*
5. Guarda cada registro en una nueva línea.

De esta forma cualquier modificación realizada en memoria queda almacenada permanentemente en ese archivo.

Modularización mediante funciones

Es pertinente aclarar, que como fue desarrollado a lo largo de la materia, se utilizó programación modular para el presente programa.

Cada tarea específica se implementó en una función independiente:

Función	Responsabilidad
<i>cargar_datos()</i>	Leer el archivo CSV
<i>guardar_datos()</i>	Guardar cambios
<i>mostrar_datos()</i>	Mostrar países
<i>agregar_pais()</i>	Alta de registros
<i>actualizar_datos()</i>	Modificación de registros
<i>buscar_pais()</i>	Búsqueda de registros
<i>filtros()</i>	Aplicación de filtros
<i>ordenar()</i>	Ordenamiento de datos
<i>estadistica()</i>	Menú de estadísticas

Esto mejora la reutilización del código y facilita el mantenimiento.

Validación de datos

Para garantizar la integridad de la información se implementaron funciones específicas de validación.

validar_nombre() verifica que:

- El dato no esté vacío.
- Contenga únicamente letras y espacios.
- Se almacene con formato adecuado.

validar_numero_entero() verifica que:

- El dato no esté vacío.
- Sea convertible a entero.
- Sea mayor que cero.

Estas validaciones evitan errores de carga y mantienen la consistencia de los datos almacenados.

Alta de países

La función `agregar_pais()` permite incorporar nuevos registros.
Durante el proceso:

1. Se solicita el nombre del país.
2. Se valida el formato.
3. Se verifica que no exista previamente.
4. Se solicitan población, superficie y continente.
5. Se genera un nuevo diccionario.
6. Se agrega a la lista mediante `append()`.
7. Se guardan los cambios en el archivo CSV.

Además, se implementaron ciclos `while True` para insistir en la carga hasta obtener valores válidos.

Actualización de datos

La función `actualizar_datos()` permite modificar la población o superficie de un país existente.
El procedimiento consiste en:

1. Buscar el país ingresado.
2. Mostrar la información encontrada.
3. Permitir seleccionar el campo a modificar.
4. Validar el nuevo valor.
5. Actualizar el diccionario correspondiente.
6. Guardar los cambios en el archivo.

Búsqueda de países

La función `buscar_pais()` implementa una búsqueda parcial que no diferencia mayúsculas de minúsculas.

La comparación se realiza mediante:

`if nombre.lower() in p["nombre"].lower():`

Al decir parcial, nos referiremos a que permite encontrar coincidencias completas o parciales.

Por ejemplo: “arg” encuentra: “Argentina”

Filtros

Se implementaron tres tipos de filtros:

- Por continente: muestra únicamente los países pertenecientes al continente indicado.
- Por población: solicita una población mínima y máxima y muestra los países comprendidos dentro del rango.
`if minimo <= p["poblacion"] <= maximo`
- Por superficie: aplica la misma lógica utilizando la superficie territorial.

Ordenamiento de datos

Para los ordenamientos se implementó manualmente el algoritmo Ordenamiento Burbuja (Bubble Sort). Este método compara elementos de la lista y los intercambia de posición cuando no están en el orden correcto. Se desarrolló:

- Ordenamiento por nombre.
- Ordenamiento por población.
- Ordenamiento por superficie.

Ejemplo del intercambio:

```
pais_temporal = paises[j] #Guarda el primer país para no perderlo durante el intercambio  
paises[j] = paises[j + 1] #Mueve el segundo país a la posición del primero  
paises[j + 1] = pais_temporal #Completa el intercambio colocando el primer país en la segunda posición
```

Además, se incorporó la posibilidad de ordenar en forma ascendente o descendente según lo requerido.

Estadísticas

Se implementaron cuatro estadísticas:

- País con mayor y menor población: se recorre la lista comparando cada país con los valores máximo y mínimo encontrados hasta el momento.
- Promedio de población: se acumulan todas las poblaciones y se divide por la cantidad de países.
promedio = pob_suma / len(paises)
- Promedio de superficie: se aplica el mismo procedimiento utilizando la superficie.

Cantidad de países por continente

Se utiliza un diccionario contador: *continentes[continente] += 1*

Permitiendo obtener la cantidad de países registrados en cada continente.

Manejo de excepciones

El programa incorpora manejo de errores mediante:

```
try:  
except ValueError:  
except Exception as e:
```

Esto permite controlar:

- Ingresos no numéricos en el menú.
- Errores inesperados durante la ejecución.

Mejorando así la robustez del sistema.

4. CONCLUSIÓN FINAL

El desarrollo del presente trabajo permitió aplicar los principales conceptos de programación estructurada vistos en la materia. Se logró implementar un sistema funcional de gestión de países que opera sobre un archivo CSV, permitiendo realizar operaciones de alta, modificación, consulta, filtrado y ordenamiento de datos.

Los recursos principales que utilizamos tanto para la lectura, manipulación y modificación de datos, como así también para la validación de entrada, fueron los vistos y sugeridos durante la cursada. Se hizo foco en la reutilización y modularización de código a través del uso de funciones principales y auxiliares, lo que facilitó tanto el mantenimiento como la coordinación del equipo de trabajo (ambos sabíamos qué tareas estaban pendientes cada vez que traíamos las actualizaciones del repositorio remoto). Asimismo, se implementaron validaciones de entrada y manejo de errores con try/except para evitar la interrupción de la ejecución.

Un ejemplo claro de esto es la función `actualizar_datos()`, utilizada para la modificación de un registro del archivo CSV. En ella observamos un bloque principal con un bucle for que itera sobre la variable local "países", que obtiene los datos mediante la función auxiliar `cargar_datos()`. Dentro del bucle, una estructura condicional compara la clave "nombre" de cada diccionario con el nombre ingresado por el usuario. En caso de encontrar coincidencia, se abre una nueva estructura condicional para decidir si se modifica la clave "población" o "superficie". Cada una de estas opciones reutiliza un bucle while y la función `validar_numero_entero()` para evitar errores de ejecución, y mediante un condicional captura el dato ingresado y lo guarda. Finalmente, a través de la función auxiliar `guardar_datos()`, se actualiza el archivo CSV con los cambios realizados.

La organización del trabajo se basó en un enfoque modular, estructurado a partir de las funcionalidades requeridas por el sistema.

En primer lugar, se definió una estructura de selección múltiple (match-case) que actúa como eje central del programa, permitiendo dirigir el flujo de ejecución según la opción elegida por el usuario en el menú principal. A partir de allí, se identificaron las tareas específicas que debía realizar cada opción, teniendo en cuenta la estructura de datos que debía utilizarse (lista de diccionarios) lo que permitió descomponer el problema en funciones independientes.

Con esta base, y teniendo claras las responsabilidades de cada opción del menú, se procedió a diseñar y codificar las funciones necesarias, distinguiendo entre:

Funciones principales: aquellas que resuelven las operaciones solicitadas por el usuario (agregar, actualizar, buscar, filtrar, ordenar, etc.).

Funciones auxiliares: aquellas que brindan soporte a las principales, como la lectura y escritura de archivos, el formateo de datos y las validaciones de entrada.

Esta estrategia permitió que ambos integrantes del equipo trabajaran de manera independiente y coordinada: mientras uno se enfocaba en el desarrollo de las funciones principales, el otro podía avanzar con las funciones auxiliares y las validaciones, asegurando que el código fuera reutilizable, claro y fácil de integrar. El uso de un repositorio compartido (GitHub) facilitó la sincronización de los avances y la resolución de conflictos de manera ágil.

5. FUENTES Y BIBLIOGRAFÍA

Material de la cátedra. (2026). Programación I - UTN TUP. Conceptos de modularización, manejo de archivos y estructuras de datos.

Python Software Foundation. (2025). The Python Standard Library
<https://docs.python.org/3/library>